

s3intel

AWS S3 Keşif ve İstihbarat Aracı — Kurulum, Tarayıcı Arayüzü ve Canlı Test Kılavuzu

AKTİF MOD — sadece whitelist'teki bucket'lar

PASİF MOD — asla gerçek bucket'a bağlanmaz

Go · cobra · aws-sdk-go-v2

make web → tarayıcıda tek tıkla çalıştır

Hazırlayan: Kadir

Tarih: 3 Ağustos 2026

Proje konumu: ~/Masaüstü/zerone/hafta-2/s3intel

Bu doküman: Aracı sıfırdan çalıştırma, whitelist/API key yapılandırma, canlı test senaryoları ve sunum/demo akışını adım adım açıklar.

1 Araç Hakkında

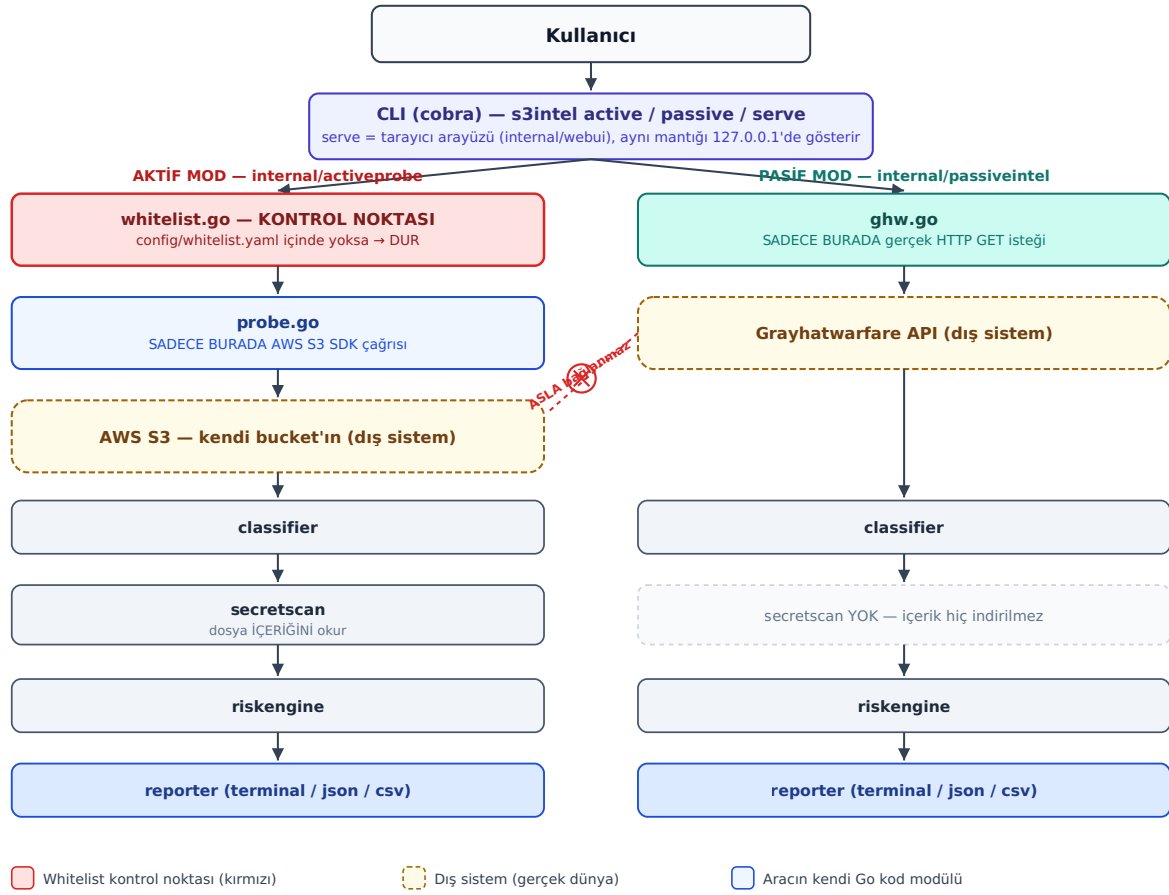
s3intel, bir dosyayı adına/uzantısına göre sınıflandıran, içeriğinde regex + Shannon entropy ile olası sızmış secret/token arayan ve bulguları 0-100 arası bir risk skoruyla önceliklendirilmiş biçimde raporlayan bir AWS S3 keşif ve istihbarat aracıdır. flaws.cloud Level 3'te elle yaptığın "listable bucket → sırayı bul" işlemini otomatikleştirir.

Kritik mimari kural: Araç kesin olarak iki moda ayrılır. **AKTİF mod** (`s3intel active`) SADECE `config/whitelist.yaml` içinde listelenen kendi test bucket'larına karşı gerçek AWS S3 API çağrısı yapar. **PASİF mod** (`s3intel passive`) SADECE grayhatwarfare API'sine (ya da API key yoksa mock veriye) HTTP sorgusu atar ve hiçbir zaman gerçek bir S3 bucket'ına bağlanmaz. Bu ayırım paket seviyesinde de fiziksel: `internal/activeprobe` hiçbir HTTP import'u içermez, `internal/passiveintel` hiçbir AWS SDK import'u içermez.

Veri akışı özeti

Aşama	Aktif mod	Pasif mod
Kaynak	Gerçek AWS S3 (<code>ListObjectsV2</code> , <code>GetObject</code>)	grayhatwarfare API / mock veri
İçerik okunur mu?	Evet — dosya indirilip taranır	Hayır — sadece dosya adı/metadate
Secret tarama	Var (regex + entropy)	Yok (içerik hiç indirilmediği için)
Sınıflandırma	Var	Var
Risk skoru	Temel risk + secret puanı	Sadece temel risk (dosya adı bazlı)

Mimari Diyagram



Ayrıntılı Mermaid versiyonu: docs/architecture.md — paket izolasyonu `grep -r "aws-sdk-go" internal/passiveintel/` ve `grep -rE "net/http|grayhatwarfare" internal/activeprobe/` komutlarıyla derleme zamanında doğrulanabilir (ikisi de boş döner).

2 Kurulum

Proje kökü: `~/Masaüstü/zerone/hafta-2/s3intel` . Terminali bu klasörde açtıktan sonra sırasıyla:

1 Bağımlılıkları indir

```
# go.mod içindeki cobra, aws-sdk-go-v2, yaml.v3 bağımlılıklarını indirir
go mod download
```

2 Derle

```
go build -o s3intel .
```

Başarılı olursa klasörde çalıştırılabilir bir `s3intel` ikili dosyası oluşur. Hiçbir hata çıktısı vermemesi gerekir.

3 Kod kalitesini doğrula (opsiyonel ama tavsiye edilir)

```
gofmt -l .      # biçim sorunu varsa dosya adlarını listeler, boş dönmeli
go vet ./...    # şüpheli kod kalıplarını denetler, boş dönmeli
```

Beklenen sonuç: Her üç komut da hatasız/çıkıntısız tamamlanır ve dizinde çalıştırılabilir bir `./s3intel` dosyası oluşur.

3 Tarayıcıda Çalıştırma (En Kolay Yol)

Flag ezberlemek ya da her seferinde komut yazmak istemiyorsan, aracın tarayıcıda çalışan basit bir arayüzü var. Bu arayüz CLI'nin yerine geçmez — aynı `internal/scanjobs` katmanını, dolayısıyla aynı whitelist kuralını ve aynı aktif/pasif ayarımını kullanır; sadece sonucu bir web sayfasında gösterir.

1 Tek komutla derle ve başlat

```
make web
```

Bu, önce `go build -o s3intel .` çalıştırır, sonra `./s3intel serve` ile **http://127.0.0.1:8080** adresinde bir sunucu açar.

2 Tarayıcında adresi aç — `http://127.0.0.1:8080` . Mod seç (Aktif/Pasif), bucket adını ya da anahtar kelimeyi yaz, "Tara" butonuna bas. Sonuç aynı sayfada, renkli bir tablo olarak görünür.

Sayfanın gerçek görünümü (aracın kendi HTML şablonundaki aynı renkler ve yapı ile, örnek/pasif "backup" araması sonucu):

127.0.0.1:8080/?mode=passive&target=backup

s3intel

PASİF — grayhatwarfare/mock

Aktif mod SADECE whitelist'teki bucket'lara gerçek AWS çağrısı yapar. Pasif mod hiçbir zaman gerçek bir bucket'a bağlanmaz.

Pasif (grayhatwarfare)

backup

Tara

Skor	Kategori	Kaynak	Secret	Dosya / Anahtar
70	env_config	passive	0	backups/backup-config.env
60	database	passive	0	dumps/backup-database.sql
55	backup	passive	0	old/backup.bak

Bu görsel, `internal/webui/server.go` içindeki gerçek HTML/CSS şablonuyla birebir aynı renk ve düzeni kullanır; veriler Bölüm 6'daki gerçek `curl` testinden alınmıştır.

Güvenlik notu: Sunucu SADECE `127.0.0.1` 'e bağlanır (`cmd/serve.go` içinde adres her zaman `127.0.0.1:<port>` olarak sabitlenir) — dışarıdan, aynı ağdaki başka bir cihazdan erişilemez. Durdurmak için terminalde `Ctrl+C` .

Alternatif: doğrudan komutla

```
./s3intel serve --port 8080
```

4 Yapılandırma

3.1 — Whitelist (aktif mod için zorunlu)

Aktif modda taranabilecek bucket'lar `config/whitelist.yaml` dosyasında listelenir. Bu, aktif kod yolunun tek giriş kapısıdır:

```
# config/whitelist.yaml
allowed_buckets:
  - "test-lab-level1"
  - "kadir-s3intel-test-bucket"
```

Buraya **sadece kendi kontrolündeki / test etmek için izinli olduğun** bucket adlarını ekle. Kendi test bucket'ını oluşturduktan sonra adını bu listeye eklemen yeterli — kod değişikliği gerekmez.

3.2 — AWS kimlik bilgileri (aktif mod için zorunlu)

s3intel, AWS SDK'nın standart yapılandırma zincirini kullanır (`aws configure` ile ayarladığın profil, ya da `AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY` ortam değişkenleri). `flaws.cloud`/Hafta 2 sürecinde zaten kurduğun AWS CLI kimlik bilgileri burada da otomatik olarak kullanılır — ayrı bir ayar gerekmez.

3.3 — grayhatwarfare API key (pasif mod için opsiyonel)

```
export GHW_API_KEY="gerçek-api-anahtarın"
```

API key olmadan da çalışır: `GHW_API_KEY` ayarlı değilse `passive` komutu otomatik olarak sabit, sahte örnek veriyle (mock) çalışır. Bu, gerçek bir grayhatwarfare hesabı olmadan da aracı uçtan uca sunabilmen için bilinçli bir tasarım kararıdır.

5 Komut Kullanımı

Bayrak	Kapsam	Açıklama
<code>--bucket</code>	active	Whitelist'te olması gereken hedef bucket adı (zorunlu)
<code>--keyword</code>	passive	grayhatwarfare arama anahtar kelimesi (zorunlu)
<code>--output</code>	active / passive	<code>terminal</code> (varsayılan) / <code>json</code> / <code>csv</code>
<code>--output-file</code>	active / passive	Belirtilirse çıktı stdout yerine bu dosyaya yazılır
<code>-v / --verbose</code>	active / passive	Ek ilerleme günlüğü basar
<code>--port</code>	serve	Web arayüzünün dinleyeceği port, varsayılan <code>8080</code> (her zaman sadece 127.0.0.1)

Örnek komutlar

```
s3intel active --bucket test-lab-level1 --output terminal
s3intel active --bucket test-lab-level1 --output json --output-file results.json
s3intel passive --keyword backup --output terminal
s3intel passive --keyword .env --output csv --output-file findings.csv
s3intel serve --port 8080 # tarayıcı arayüzü, bkz. Bölüm 3
```

6 Canlı Test Senaryosu (Adım Adım)

Bu bölüm, aracı sunum/canlı savunma sırasında hangi sırayla göstereceğini adım adım anlatır. Her adımda "ne olmasını bekle" belirtilmiştir; aşağıdaki terminal blokları bu kılavuz hazırlanırken **gerçekten çalıştırılmış** komutların ham çıktılarıdır.

Adım 1 — Test suite'i çalıştır (araç doğru çalışıyor mu?)

1

```
go test ./...
```

```
$ go clean -testcache && go test ./...

?      s3intel [no test files]
ok      s3intel/cmd      0.013s
ok      s3intel/internal/activeprobe 0.012s
ok      s3intel/internal/classifier  0.010s
ok      s3intel/internal/passiveintel 0.012s
ok      s3intel/internal/reporter    0.008s
ok      s3intel/internal/riskengine  0.007s
ok      s3intel/internal/scanjobs    0.011s
ok      s3intel/internal/secretscan  0.010s
ok      s3intel/internal/webui      0.007s
```

Tüm paketlerde `ok` görülmeli — dokuz paket, hepsi kendi `_test.go` dosyasına sahip. `[no test files]` sadece `main.go` 'nun bulunduğu kök paket için normaldir (mantık tamamen `cmd/` ve `internal/` altındadır). `scanjobs` ve `webui`, bir önceki sürümden bu yana eklenen tarayıcı arayüzü katmanına ait.

Adım 2 — Tarayıcı arayüzünü canlı başlat

2

```
make web
# ayrı bir terminalde dene:
curl -s "http://127.0.0.1:8080/?mode=passive&target=backup" | grep -A1 score
```

```
$ curl -s "http://127.0.0.1:8080/?mode=active&target=not-in-whitelist" | grep -A1 'class="error"'

<div class="error"><b>Hata:</b> bucket not-in-whitelist is not in the whitelist – active probing refused</div>
```

Anlatım notu: "Web arayüzü de arkada tam olarak aynı whitelist kontrolünü kullanıyor — sayfada sadece daha okunaklı bir hata kutusu olarak gösteriliyor." Bölüm 3'teki tarayıcı görselinde aynı sayfanın pasif modda nasıl görüldüğünü görebilirsin.

Adım 3 — PASİF modu CLI'den, API key olmadan göster (mock veri)

3

```
unset GHW_API_KEY
./s3intel passive --keyword test --output terminal
```

```
$ ./s3intel passive --keyword test --output terminal

SKOR      KATEGORİ    KAYNAK    SECRET    ANAHTAR/DOSYA
70  env_config  passive  0          backups/test-config.env
60  database   passive  0          dumps/test-database.sql
55  backup     passive  0          old/test.bak
```


Anlatım notu: "GHW_API_KEY ayarlı değil, ama araç hâlâ çalışıyor çünkü mock kaynağa düşüyor. Bu üç sonuç grayhatwarfare'den değil, sabit örnek veriden geliyor — gerçek bir API isteği hiç gitmedi." Terminaldeki kırmızı/sarı renk, skor eşiklerine göre otomatik belirleniyor (≥ 70 kırmızı, ≥ 40 sarı, altı yeşil).

Adım 4 — AKTİF modda whitelist reddini CLI'den göster (AWS'ye hiç dokunmadan)

4

```
./s3intel active --bucket not-in-whitelist
```

```
$ ./s3intel active --bucket not-in-whitelist
Error: bucket not-in-whitelist is not in the whitelist – active probing refused
Usage:
  s3intel active [flags]
...
exit code: 1
```

Anlatım notu: "Bu bucket whitelist'te yok, bu yüzden AWS'ye tek bir çağrı bile gitmiyor — hata whitelist.go içinde, S3 istemcisi hiç oluşturulmadan dönüyor." Bu davranış `internal/activeprobe/probe_test.go` içindeki `TestEnumerateBucket_RejectsNonWhitelistedBucket` testiyle de kanıtlanmıştır (mock S3 istemcisinin hiç çağrılmadığı doğrulanır).

Adım 5 — Kendi test bucket'ınla AKTİF modu gerçek AWS'ye karşı çalıştır

Bu adım, flaws.cloud sürecinde kurduğun AWS CLI kimlik bilgilerini ve kendi oluşturacağın bir test bucket'ını kullanır.

1

Kendi test bucket'ını oluştur (bölge ve isim kendine ait olmalı, isim global olarak benzersiz olmak zorunda):

```
aws s3 mb s3://kadir-s3intel-test-bucket --region eu-central-1
```

2

İçine "sızıntı" gibi görünen birkaç test dosyası yükle — örneğin `testdata/sample_secrets.txt` dosyasını farklı adlarla:

```
aws s3 cp testdata/sample_secrets.txt s3://kadir-s3intel-test-bucket/config/.env
aws s3 cp testdata/sample_secrets.txt s3://kadir-s3intel-test-bucket/backups/site-backup.zip
echo "readme" > /tmp/readme.txt
aws s3 cp /tmp/readme.txt s3://kadir-s3intel-test-bucket/readme.txt
```

3

Bucket adının whitelist'te olduğunu doğrula — `config/whitelist.yaml` içinde `kadir-s3intel-test-bucket` zaten örnek olarak duruyor; farklı bir isim kullandıysan burayı güncelle.

4

Aracı çalıştır

```
./s3intel active --bucket kadir-s3intel-test-bucket --output terminal
```

Beklenen: `config/.env` dosyası yüksek skorla (AKIA deseni + `env_config` kategorisi üst üste bindiği için) en üstte, `backups/site-backup.zip` orta skorla, düz metin `readme.txt` en altta düşük skorla listelenir — **azalan sırada**.

Unutma: Bu adım gerçek AWS çağrısı yapar ve gerçek AWS maliyeti/izin gerektirir. Sadece kendi whitelist'teki, kendi oluşturduğun bucket'a karşı çalıştır — asla başka birinin bucket'ına karşı denemeye çalışma.

Adım 6 — JSON ve CSV çıktısını göster

```
$ ./s3intel passive --keyword backup --output json
```

```
[
  {
    "Key": "backups/backup-config.env",
    "Source": "passive",
    "Category": "env_config",
    "BaseRisk": 70,
    "Secrets": null,
    "Score": 70
  },
  {
    "Key": "dumps/backup-database.sql",
    "Source": "passive",
    "Category": "database",
    "BaseRisk": 60,
    "Secrets": null,
    "Score": 60
  }
]
```

```
$ ./s3intel passive --keyword .env --output csv
score,category,source,secret_count,key
70,env_config,passive,0,backups/.env-config.env
70,env_config,passive,0,dumps/.env-database.sql
70,env_config,passive,0,old/.env.bak
```

Bu iki format, bulguları başka bir araca (Excel, jq, başka bir script) aktarmak için kullanılır; `--output-file` ile doğrudan dosyaya yazdırılabilir.

7 Çıktının Yorumlanması

Risk skoru renk eşikleri (terminal çıktısı)

Renk	Skor aralığı	Anlamı
Kırmızı	70 - 100	Yüksek risk — kritik kategori ve/veya yüksek güvenilirlikte secret bulundu
Sarı	40 - 69	Orta risk — dikkatle incelenmeli
Yeşil	0 - 39	Düşük risk — muhtemelen zararsız

Skor nasıl hesaplanıyor?

skor = classifier temel riski + Σ (bulunan her secret'in confidence puanı) , 100'e sıkıştırılır (cap).

Confidence	Ek puan	Ne zaman verilir
high	+40	Kesin regex eşleşmesi (AKIA anahtarı, private key header, aws_secret_access_key)
medium	+20	Bağlamsal regex eşleşmesi (api_key= , password=)
low	+10	Regex'e yakalanmayan ama entropy eşliğinin (≥ 4.5 bit/karakter) üzerindeki string

Kategoriler (classifier)

Kategori	Temel risk	Örnek desenler
private_key	85	.pem , .key , id_rsa
credentials	80	credential , .aws/credentials
env_config	70	.env
database	60	.sql , .dump , .db
backup	55	backup , .bak , .old
archive	30	.zip , .tar , .gz
other	10	eşleşmeyen her şey

8 Sunum / Canlı Savunma Akışı Önerisi

~5-6 dakikalık demo bölümü için önerilen sıra (Bölüm 6'daki adımlarla birebir eşleşir):

1. **Mimari diyagramı göster** (bu dokümanın 1. bölümü) — "iki mod var, biri gerçek AWS'ye dokunur biri dokunmaz" cümlesiyle aç.
2. **go test ./... çalıştır** — "araç test edilmiş, whitelist reddi de dahil her modül kendi testine sahip" de.
3. **Tarayıcı arayüzünü aç** (`make web` , `127.0.0.1:8080`) — asıl demoyu buradan yap: pasif modda "backup" ara, sonra aktif modda whitelist dışı bir bucket yazıp hatanın tarayıcıda nasıl görüldüğünü göster. Bu, komut satırı bilmeyen bir izleyiciye bile aracı anlatmayı kolaylaştırır.
4. **Aynı whitelist reddini bu sefer CLI'den göster** (`active --bucket not-in-whitelist`) — "arka planda ikisi de aynı kodu, aynı whitelist kontrolünü kullanıyor, sadece görünüm farklı" vurgusunu yap.
5. **Kendi whitelist'teki bucket'ına karşı gerçek taramayı göster** (CLI ya da tarayıcıdan) — flaws.cloud Level 3'ü elle yaptığın işlemi bu aracın otomatikleştirdiğini bağla.
6. **JSON/CSV çıktısını göster** — "bulgular başka araçlara aktarılabilir" ile kapat.

Sorulabilecek soru: "Whitelist kontrolü nerede yapılıyor?" — Cevap: `internal/activeprobe/probe.go` içindeki `EnumerateBucket` fonksiyonunun ilk satırı; kontrol ile AWS istemcisinin oluşturulması arasında başka hiçbir kod yok, bu yüzden reddedilirse istemci hiç kurulmuyor.

9 Sorun Giderme

Belirti	Olası neden	Çözüm
<code>bind: address already in use</code>	<code>make web</code> zaten çalışıyor ya da 8080 portu doluyken tekrar başlatıldı	Önceki sunucuyu <code>Ctrl+C</code> ile durdur, ya da <code>./s3intel serve --port 8081</code> gibi farklı bir port kullan
<code>bucket X is not in the whitelist</code>	Bucket adı <code>config/whitelist.yaml</code> 'da yok	Bucket adını <code>allowed_buckets</code> listesine ekle
AWS yapılandırması yüklenemedi	AWS kimlik bilgileri ayarlı değil	<code>aws configure</code> çalıştır ya da <code>AWS_ACCESS_KEY_ID / AWS_SECRET_ACCESS_KEY</code> ortam değişkenlerini ayarla
<code>bucket X listelenemedi: ... AccessDenied</code>	Kullanılan IAM kullanıcısının bucket'a izni yok	IAM policy'de <code>s3:ListBucket / s3:GetObject</code> izinlerini kontrol et
grayhatwarfare API beklenmeyen durum kodu döndürdü	Geçersiz/süresi dolmuş <code>GHW_API_KEY</code>	Key'i kontrol et, ya da <code>unset GHW_API_KEY</code> ile mock moda dön
bilinmeyen output formatı	<code>--output</code> yanlış yazıldı	Sadece <code>terminal</code> , <code>json</code> , <code>csv</code> geçerli

Hızlı sağlık kontrolü

```
go build -o s3intel . && go test ./... && ./s3intel passive --keyword test
```

Bu tek satır, derlemenin, tüm testlerin ve pasif modun (mock veriyle) çalıştığını saniyeler içinde doğrular.